

Ad Click Aggregation

రోజుకి 1.7 PB events నుంచి 72 GB సమాధానాలు – **24,240 రెట్ల కుంచింపు**. కూడటం సులభమైన భాగం. కష్టమైనది – ఆ సంఖ్యల మీద **advertiser కి బిల్లు వేస్తారు**.

మూడు విరుపులు. మొదటిది – at-least-once delivery తో లెక్క **17.57% ఉబ్బుతుంది**, మరియు అది నేరుగా అదనపు బిల్లు. రెండోది – ఒక click **రెండు గంటల తర్వాత** రావచ్చు; ఎంతసేపు వేచి ఉన్నా తోక ముగియదు. మూడోది – "ఎంతమంది ప్రత్యేక వాడుకరులు" అనే సంఖ్యలను కలిపితే **405% ఉబ్బు**.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output, ప్రతి శాతం నిజంగా `node` `lo run` చేసినదే.

ఈ **PROBLEM** ని **ANALYTICS PROBLEM** గా చూస్తే మీరు తప్పు **DOC** చదువుతున్నారు

"రోజుకి కోట్లాది events ని కూడు" అనేది ఒక పరిష్కరించిన problem – Kafka, Flink, ఒక column store, ముగిసింది.

ఈ problem కష్టం ఎందుకంటే ఆ సంఖ్య ఒక **dashboard** కాదు – అది ఒక **invoice**. 1% ఎక్కువ లెక్కపెడితే మీరు advertiser ని మోసం చేస్తున్నారు. 1% తక్కువ లెక్కపెడితే మీరు publisher కి చెల్లించడం లేదు.

కాబట్టి "దాదాపు సరైనది" సరిపోదు, మరియు "ఖచ్చితంగా సరైనది" ఈ వేగంతో సాధ్యం కాదు.

ఈ doc మొత్తం ఆ రెండింటి మధ్య ఎక్కడ నిలబడాలో గురించి – మరియు ప్రతి ఎంపికకీ ఒక కొలిచిన సంఖ్య ఉంది.

విషయ సూచిక (Table of Contents)

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు
2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 – మొదటి విరుపు: రెండుసార్లు లెక్క

3. Step – ప్రతి click కి ఒకటి కలుపు
4. మొదటి విరుపు – లెక్క 17.57% ఉబ్బింది
5. Step – dedup విండో · మరియు దాని నిజమైన ఖర్చు

Part 3 – రెండో విరుపు: ఆలస్యంగా వచ్చినవి

6. Step – నిమిషానికి ఒక bucket
7. రెండో విరుపు – తోక ఎప్పటికీ ముగియదు
8. Step – watermark · మరియు ఆలస్యమైన దాన్ని ఏమి చేయాలి

Part 4 – మూడో విరుపు: ప్రత్యేక వాడుకరులు

9. Step – "ఎంతమంది చూశారు?"
10. మూడో విరుపు – ప్రత్యేక సంఖ్యలను కలపలేము
11. Step – HyperLogLog · మరియు అది ఏమి చేయలేదు

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code · 5.6 లక్షల events · mutation testing
13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

14. నోటీస్ చెప్పాల్సిన English script

15. Follow-ups – fraud, lambda vs kappa, సరిదిద్దడం

16. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు ఏది సమస్య కాదు

Clicks మరియు impressions లెక్కపెట్టడం. ఒక్కో ad కి, ఒక్కో నిమిషానికి. Advertisers dashboard lo దాదాపు తక్షణమే చూడాలి, మరియు నెలాఖరున ఆ సంఖ్యల మీదే **బిల్లు** వస్తుంది.

సెకనుకి 1 మిలియన్ clicks · 100 మిలియన్ impressions

event రకం	సెకనుకి	రోజుకి events	రోజుకి ముడి data
clicks	1M	86 బిలియన్	17 TB
impressions	100M	8,640 బిలియన్	1.7 PB

కానీ సమాధానం: ఒక్కో ad కి, ఒక్కో నిమిషానికి ఒక వరుస
1 మిలియన్ ads × 1,440 నిమిషాలు = 1,440 మిలియన్ వరుసలు → 72 GB/రోజు

కుంచింపు: 1.7 PB → 72 GB = 24,240 రెట్లు

24,240 రెట్లు కుంచింపు – అందుకే ఇది ఒక SCALE PROBLEM కాదు

1.7 PB ని 72 GB కి తగ్గించడం అంటే ఒక stream processor ప్రతి event ని **ఒక్కసారి** చూసి, ఒక counter పెంచి, మర్చిపోవడం.

Kafka సెకనుకి కోట్ల events నిర్వహిస్తుంది (HLD 13 lo కొలిచాను – ముద్ద చేస్తే machine కి 2.8 లక్షలు). 100 machines సరిపోతాయి.

కాబట్టి "ఎలా కూడాలి" అనేది ప్రశ్న కాదు. ప్రశ్న – ఏమి కూడాలి, మరియు ఎప్పుడు ఆపాలి.

ఆ మూడు నిజమైన ప్రశ్నలు:

- ఒక click ని రెండుసార్లు లెక్కపెడితే → advertiser కి రెండుసార్లు బిల్లు (\$4)
- ఒక click ఆలస్యంగా వస్తే → ఏ నిమిషానికి చెందుతుంది? (\$7)
- "ఎంతమంది ప్రత్యేక వాడుకరులు" → ఆ సంఖ్యలను కలపలేము (\$10)

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	ఎందుకు అడుగుతున్నాం
ఈ సంఖ్యల మీద బిల్లు వేస్తారా , లేక dashboard మాత్రమేనా?	బిల్లు అయితే "దాదాపు సరైనది" సరిపోదు – §5
ఎంత ఆలస్యం ఆమోదయోగ్యం – సెకన్లూ, నిమిషాలా?	§8 – ఇదే watermark ని నిర్ణయిస్తుంది
Client retry చేస్తుందా?	అవును అంటే §4 – మరియు ప్రతి client చేస్తుంది
Event కి ఒక ప్రత్యేక id ఉందా?	లేకపోతే dedup అసాధ్యం. ఇది ఒక client-side design నిర్ణయం
"ప్రత్యేక వాడుకరులు" కావాలా?	అవును అంటే §10 – మరియు అది మొత్తం storage ని మారుస్తుంది
ఎంత వెనక్కి సరిదిద్దగలగాలి?	§15 – ఒక రోజు, ఒక నెలా? ఇది మీ నిల్వని నిర్ణయిస్తుంది
Bot clicks ని ఏమి చేయాలి?	§15 – మరియు అవి మీ లెక్కలో ఇప్పటికే ఉన్నాయి

ఒక ప్రశ్న అడిగితే మీరు దీన్ని నిజంగా నడిపారని తెలుస్తుంది

"ఈ సంఖ్య ఎప్పుడు 'తుది' అవుతుంది – మరియు దాని తర్వాత మారగలదా?"

ఎందుకంటే §7 lo కొలుస్తాం: 15 నిమిషాలు వేచి ఉన్నా **0.16% events** ఇంకా రాలేదు. 2 గంటలు వేచి ఉన్నా **0.01%** రాలేదు. **ఆ తేక ఎప్పటికీ ముగియదు.**

కాబట్టి "తుది సంఖ్య" అనేది ఒక **నిర్ణయం**, ఒక సహజ క్షణం కాదు. మీరు ఎప్పుడు ఆపాలో ఎంచుకోవాలి, మరియు ఆ తర్వాత వచ్చిన వాటిని ఏమి చేయాలో కూడా.

ఈ ప్రశ్న అడగడం అంటే మీకు తెలుసు: **ఇది ఒక కూడిక problem కాదు, ఒక సమయ problem.**

Part 2 – మొదటి విరుపు: రెండుసార్లు లెక్క

3. Step – ప్రతి click కి ఒకటి కలుపు

సరళమైన pipeline:

```
browser → click event → Kafka → stream processor → counter[ad][minute]++
```

Kafka **at-least-once** ఇస్తుంది (HLD Deep 13 §15 lo కొలిచాను – 40 లక్షల సందేశాలకి 84,711 నకళ్ళు). అంటే ack పోతే producer మళ్ళీ పంపుతాడు.

Analytics dashboard కి ఇది ఫరవాలేదు. **ఒక invoice కి కాదు.**

4. మొదటి విరుపు – లెక్క 17.57% ఉబ్బింది

2 లక్షల నిజమైన clicks, ఒక్కో దానికి గరిష్ఠంగా 5 retries, retry ఒక్కో దానికీ 1-120 సెకన్ల తర్వాత:

retry రేటు	పంపినవి	dedup లేకుండా లెక్క	ఉబ్బు %	బిల్లు తేడా
0.5%	2,00,971	2,00,971	0.49%	₹1,942
2.0%	2,04,099	2,04,099	2.05%	₹8,198
5.0%	2,10,583	2,10,583	5.29%	₹21,166
15.0%	2,35,142	2,35,142	17.57%	₹70,284

0.5% RETRY రేటు = 0.49% అదనపు బిల్లు • మరియు ఇది 2 లక్షల CLICKS మాత్రమే

రేజికి 86 బిలియన్ clicks. 0.5% అంటే రేజికి **43 కోట్ల** నకిలీ clicks.

మరియు ఇది ఒక bug కాదు – ఇది delivery హామీ **సరిగ్గా పనిచేయడం**. At-least-once అంటే "కనీసం ఒకసారి", మరియు ఆ మాట నిజాయితీగా చెబుతోంది.

మీరు at-least-once వ్యవస్థ మీద exactly-once లెక్క నిర్మించలేరు – ఆ పనిని application చేయాల్సిందే.

5. Step – dedup విండో • మరియు దాని నిజమైన ఖర్చు

ప్రతి event కి ఒక **ప్రత్యేక id** (browser lo తయారవుతుంది), మరియు processor ఇప్పటికే చూసిన ids ని గుర్తుపెట్టుకుంటుంది.

కానీ అన్నిటిని గుర్తుపెట్టుకోలేము. కాబట్టి ఒక **విండో** – ఎన్ని సెకన్ల విలువైన ids ఉంచాలి?

dedup విండో ఎంత ఉంచాలి? (retry రేటు 5%, retry ఆలస్యం గరిష్ఠంగా 120 సె)

విండో (సె) లెక్క మిగిలిన నకళ్ళు ఉబ్బు % గుర్తుపెట్టుకున్న ids	
– 2,10,583 10,583 5.292% 0	
10 2,09,590 9,590 4.795% 12	
30 2,07,741 7,741 3.870% 10,733	
60 2,05,161 5,161 2.580% 21,358	
300 2,00,000 0 0.000% 1,00,485	
3600 2,00,000 0 0.000% 2,00,000	

నియమం సులభమైనది: DEDUP విండో $\geq$ గరిష్ఠ RETRY ఆలస్యం

ఈ ప్రయోగంలో retry గరిష్ఠంగా 120 సెకన్ల తర్వాత వస్తుంది. విండో 60 సె → ఇంకా **2.58% ఉబ్బు**. విండో 300 సె → **సరిగ్గా సున్నా**.

మరియు 300 తర్వాత ఏమీ మెరుగుపడదు – 3600 కూడా అదే సున్నా, కేవలం రెట్టింపు memory.

కాబట్టి ఇది ఒక ఏకపక్ష సంఖ్య కాదు: **మీ producer యొక్క retry విధానం మీ dedup విండోని నిర్ణయిస్తుంది**. ఆ రెండూ వేర్వేరు బృందాలు రాసినవి అయితే, ఇది ఒక **చెప్పని ఒప్పందం** – మరియు అది మారినప్పుడు మౌనంగా విరుగుతుంది.

ఇప్పుడు ఆ memory యొక్క నిజమైన ఖర్చు – సెకనుకి 10 లక్షల clicks తో:

విండో గుర్తుపెట్టుకోవాల్సిన ids memory ఎన్ని machines (64 GB)	
60 సె 60M 1.0 GB 1	
300 సె 300M 4.8 GB 1	
900 సె 900M 14.4 GB 1	
1 గం 3,600M 57.6 GB 1	
1 రోజు 86,400M 1382.4 GB 22	

"ఒక రోజు DEDUP చేద్దాం" అనే జవాబుకి ఒక ధర ఉంది – 22 MACHINES

మరియు అది కేవలం "ఇది ఒక నకలా?" అనే ప్రశ్నకి జవాబు చెప్పడానికి.

5 నిమిషాల విండో **4.8 GB** – ఒక్క machine. అది retry ఆలస్యం కంటే ఎక్కువైతే సరిపోతుంది.

కాబట్టి సరైన నిర్ణయం "ఎంత దీర్ఘంగా dedup చేయగలం" కాదు – **"producer ఎంతసేపటి తర్వాత retry చేయడం మానేస్తాడు"**. ఆ ఒక్క సంఖ్య తెలిస్తే మిగతాదంతా వస్తుంది.

Part 3 – రెండో విరుపు: ఆలస్యంగా వచ్చినవి

6. Step – నిమిషానికి ఒక bucket

ఇప్పుడు నకళ్ళు పోయాయి. తర్వాతి ప్రశ్న: ఒక click ఏ నిమిషానికి చెందుతుంది?

సహజమైన జవాబు: అది వచ్చిన నిమిషానికి.

```
counter[ad][Math.floor(Date.now()/60000)]++; // వచ్చిన సమయం
```

ఇది తప్పు, మరియు కారణం స్పష్టం: ఒక వాడుకరి metro lo ad మీద click చేసి, signal వచ్చాక ఆ event **10 నిమిషాల తర్వాత** పంపుతాడు. ఆ click 10:00 కి జరిగింది, 10:10 కి వచ్చింది.

కాబట్టి **event సమయం** వాడాలి:

```
counter[ad][Math.floor(event.eventTime/60000)]++; // జరిగిన సమయం
```

ఇప్పుడు ఒక కొత్త సమస్య: **10:00 నిమిషం ఎప్పుడు "ముగిసింది" అని చెప్పాలి?**

7. రెండో విరుపు – తోక ఎప్పటికీ ముగియదు

5 లక్షల clicks, 6 గంటలు, వాస్తవికమైన ఆలస్య పంపిణీ (90% తక్షణమే, 0.1% రెండు గంటల వరకు):

watermark	ఆలస్యంగా వచ్చినవి	%	"ముగిసింది" అనడానికి ఆలస్యం
0 సె	29,950	5.990%	0.0 నిమిషాలు
5 సె	26,760	5.352%	0.1 నిమిషాలు
30 సె	14,965	2.993%	0.5 నిమిషాలు
60 సె	9,681	1.936%	1.0 నిమిషాలు
300 సె	7,152	1.430%	5.0 నిమిషాలు
900 సె	798	0.160%	15.0 నిమిషాలు
7200 సె	51	0.010%	120.0 నిమిషాలు

2 గంటలు వేచి ఉన్నా 0.01% ఇంకా రాలేదు - మరియు అది ఎప్పటికీ సున్నా కాదు

ఒక్క నిమిషం వేచి ఉంటే 1.936% పోతాయి. 15 నిమిషాలు వేచి ఉంటే 0.16%. 2 గంటలు వేచి ఉంటే 0.01%.

ప్రతిసారీ 10 రెట్లు మెరుగు, ప్రతిసారీ **10 రెట్లు ఎక్కువ ఆలస్యం**. మరియు తోక ముగియదు - ఒక ఫోన్ ఒక వారం తర్వాత తిరిగి online కి రావచ్చు.

"పూర్తి సంఖ్య" అనేది ఉనికిలో లేదు. కేవలం "ఎంతసేపు వేచి ఉన్నాం" అనేది ఉంది.

8. Step – watermark · మరియు ఆలస్యమైన దాన్ని ఏమి చేయాలి

Watermark = "ఈ క్షణం కంటే ముందు జరిగిన events అన్నీ వచ్చేశాయని మేము అనుకుంటున్నాము". ఒక window ముగిసిన watermark సెకన్ల తర్వాత దాన్ని మూసేస్తాం.

ఆ తర్వాత వచ్చిన event కి మూడు ఎంపికలు, మరియు మూడూ వేర్వేరు రకంగా తప్పు:

ఆలస్యంగా వచ్చిన event ని ఏమి చేయాలి? (watermark 60 సె)

విధానం	మొత్తం తప్పు	తప్పు %	తప్పు నిమిషంలో	తప్పు *గంట*లో
పడేయడం (drop)	9,681	1.936%	-	-
ఇప్పటి window lo వేయడం 1,798	2,231	0.446%	9,681	-
పాత window ని సరిదిద్దడం	0	0.000%	-	-

రెండే వరుస ఒక ఉచ్చు - అది ప్రతి సగటు కొలతలోనూ మెరుగ్గా కనిపిస్తుంది

"ఇప్పటి window lo వేయడం" యొక్క మొత్తం తప్పు **0.446%** - "పడేయడం" యొక్క 1.936% కంటే నాలుగు రెట్లు తక్కువ.

ఎందుకంటే ఆ events పోలేదు - అవి **పక్క window కి** వెళ్ళాయి, మరియు ఒక window కి వచ్చిన నష్టం ఇంకొక window యొక్క లాభంతో కొంత సరిపోతుంది.

కానీ మూడో నిలువు వరుస చూడండి: **9,681 events తప్పు నిమిషంలో ఉన్నాయి**, మరియు **1,798 తప్పు గంటలో**.

Campaign ఆ గంటలో ముగిసి ఉంటే - ఆ 1,798 clicks ఒక **ముగిసిన campaign కి** బిల్లు అవుతాయి. లేదా ఆ రోజు budget అయిపోయాక వచ్చినట్టు కనిపిస్తాయి.

ఒక్క సగటు తప్పు శాతం ఈ నష్టాన్ని పూర్తిగా దాచేస్తుంది.

కాబట్టి సరైన జవాబు మూడోది - మరియు దానికి ఒక నిర్మాణాత్మక ధర ఉంది

పాత window ని సరిదిద్దడం - తప్పు సున్నా. కానీ దీనికి మీ నిల్వ **మార్చదగినది** కావాలి: "10:00 నిమిషానికి 5,000 clicks" అనే వరుసని రెండు గంటల తర్వాత 5,003 కి మార్చగలగాలి.

అంటే మీరు advertiser కి ఇచ్చే సంఖ్య **తాత్కాలికమైనది**, మరియు అది product lo కనిపించాలి - "ఈ సంఖ్య ఇంకా స్థిరపడలేదు" అని.

ఆచరణలో రెండు దశలు: **వేగవంతమైన తాత్కాలిక సంఖ్య** (సెకన్లలో, watermark చిన్నది), మరియు **ఒక రోజు తర్వాత స్థిరపడే తుది సంఖ్య** (బిల్లుకి అదే). \$15 lo దాని గురించి.

Part 4 – మూడో విరుపు: ప్రత్యేక వాడుకరులు

9. Step – "ఎంతమంది చూశారు?"

Advertisers clicks మాత్రమే అడగరు. వాళ్ళు అడిగేది: "ఎంతమంది ప్రత్యేక వ్యక్తులు నా ad చూశారు?" (reach).

సహజమైన అమలు: ఒక్కో ad కి ఒక Set .

```
uniques[ad].add(userId);
```

ఇది స్పష్టంగా సరైనది. మరియు అది రెండు రకాలుగా విరుగుతుంది.

10. మూడో విరుపు – ప్రత్యేక సంఖ్యలను కలపలేము

మొదటి విరుపు: మీ processing shards మద పంచబడి ఉంది. ఒక్కో shard తన ప్రత్యేక వాడుకరులను లెక్కిస్తుంది. వాటిని కలపాలి.

1. ఒక ad ని 8 shards lo లెక్కిస్తే – ప్రత్యేక వాడుకరుల సంఖ్యలు కలపవచ్చా?

నిజమైన ప్రత్యేక వాడుకరులు : 49,983

ఒక్కో shard యొక్క సంఖ్యల మొత్తం : 2,52,631

ఉబ్బు : 405% ← clicks ని కలపొచ్చు, ప్రత్యేక వాడుకరులని కాదు

49,983 నిజం • 2,52,631 కలిపితే • 405% ఉబ్బు

కారణం సులభమైనది: ఒకే వాడుకరి 8 shards లో ఉంటే, 8 సార్లు లెక్కించబడతాడు.

మరియు ఇది ఒక పరిష్కరించదగిన bug కాదు – ఇది ఒక గణిత నిజం. Clicks ఒక కూడదగిన కొలత ($|A| + |B|$). ప్రత్యేక వాడుకరులు కాదు ($|A \cup B| \neq |A| + |B|$).

కొన్ని కొలతలు కూడబడతాయి. కొన్ని కూడబడవు. ఆ తేడామే ఈ section మొత్తం.

రెండో విరుపు: కలపాలంటే **పూర్తి sets** పంపాలి, సంఖ్యలు కాదు. అవి ఎంత పెద్దవి?

2. ఖచ్చితంగా లెక్కించాలంటే ఎంత memory?

ప్రత్యేక వాడుకరులు | ఖచ్చితమైన set | HLL (p=14) | HLL తప్పు %

1,000	0.02 MB	16 KB	0.20%
10,000	0.15 MB	16 KB	0.11%
1,00,000	1.53 MB	16 KB	0.75%
10,00,000	15.26 MB	16 KB	1.01%
1,00,00,000	152.59 MB	16 KB	0.99%

ఒక్కో ad కి 152 MB. 10 లక్షల ads ఉంటే అది **152 TB** – మరియు అది ఒక్క రోజుకీ, ఒక్క నిమిషానికి కాదు.

11. Step – HyperLogLog · మరియు అది ఏమి చేయలేదు

HyperLogLog ఒక వింత ఉపాయం మీద ఆధారపడుతుంది: ఒక హాష్ యొక్క ముందు ఎన్ని సున్నాలు ఉన్నాయో చూస్తే, ఎన్ని వేర్వేరు విలువలు చూశామో **ఊహించవచ్చు**. 20 సున్నాలు కనిపించాయంటే దాదాపు 2^{20} వేర్వేరు విలువలు చూసి ఉంటాం.

```
add(x) {
  const h = h32(x), i = h >>> (32 - this.p);          // ఏ register
  let w = (h << this.p) >>> 0, z = 1;
  while (z <= 32 - this.p && !(w & 0x80000000)) { z++; w = (w << 1) >>> 0; }
  if (z > this.R[i]) this.R[i] = z;                    // ← గరిష్ఠం
}
merge(o) { for (let i = 0; i < this.m; i++) if (o.R[i] > this.R[i]) this.R[i] = o.R[i]; }
```

ఆ గరిష్ఠం అనే పదమే **MERGE** ని సాధ్యం చేస్తుంది

ప్రతి register "ఇప్పటిదాకా చూసిన అత్యధిక" విలువని ఉంచుతుంది. గరిష్ఠం అనేది **idempotent మరియు commutative** – కాబట్టి రెండు HLL లను కలపడం అంటే register వారీగా గరిష్ఠం తీసుకోవడం.

అంటే ఏ క్రమంలో, ఎన్నిసార్లు కలిపినా అదే జవాబు. ఒకే event రెండు shards lo ఉన్నా ఫరవాలేదు.

ఇది §10 యొక్క 405% సమస్యని పూర్తిగా తొలగిస్తుంది – మరియు అది **ఒక ఉజ్జాయింపు కావడం వల్ల** కాదు, **merge ఒక గరిష్ఠం కావడం వల్ల**.

ఖర్చు – ఎంత పెద్దది కావాలి:

3. HLL ఎంత పెద్దది కావాలి? (10 లక్షల ప్రత్యేక వాడుకరులు)

p	registers	memory	తప్పు %	సిద్ధాంత తప్పు %
8	256	0 KB	6.05%	6.50%
10	1,024	1 KB	4.30%	3.25%
12	4,096	4 KB	2.25%	1.63%
14	16,384	16 KB	0.40%	0.81%
16	65,536	64 KB	0.24%	0.41%

కొలిచిన తప్పు సిద్ధాంతం (1.04/√m) తో సరిపోతుంది. **16 KB తో 1% లోపు** – ఖచ్చితమైన set కంటే **9,500 రెట్లు** తక్కువ.

కానీ HLL మూడు పనులు చేయలేదు, మరియు మూడూ నిజమైన ఉత్పత్తి అవసరాలు

1. తనివేయలేము. "A చూసినవాళ్ళలో B చూడనివాళ్ళు ఎంతమంది?" – HLL కి జవాబు లేదు. కలపగలదు, తనివేయలేదు. (దీనికి వేరే నిర్మాణం కావాలి.)

2. "ఈ వాడుకరి ఇందులో ఉన్నాడా?" అని అడగలేము. HLL ఒక సంఖ్య మాత్రమే ఇస్తుంది, సభ్యత్వం కాదు. Retargeting కి అది పనికిరాదు.

3. 1% తప్పు అనేది సాపేక్ష తప్పు. 10 లక్షలకి అది 10,000 – ఒక సంఖ్యగా పెద్దది. బిల్లు reach మీద ఆధారపడి ఉంటే, ఆ 1% ని ఎవరు భరిస్తారో **ఒప్పందంలో** రాయాలి.

కాబట్టి సరైన design: **clicks ఖచ్చితంగా** (అవి కూడబడతాయి, మరియు బిల్లు వాటి మీద), **reach ఉజ్జాయింపుగా** (HLL, మరియు అది ఒక నివేదిక సంఖ్య).

Part 5 – నిరూపణ మరియు ప్రదర్శన

12. మొత్తం code • 5.6 లక్షల events • mutation testing

అయిదు నియమాలు: ప్రతి event యొక్క ఫలితం (లెక్క/నకలు/ఆలస్యం) నిజంతో ఏకీభవించాలి, పరిరక్షణ – వచ్చినవి = లెక్కించినవి + నకళ్ళు + ఆలస్యమైనవి, ఒక్కో window యొక్క clicks ఖచ్చితంగా సరైనవి, shards ని కలిపితే clicks కేంద్రంతో సమానం, కలిపిన uniques కేంద్రం యొక్క uniques తో సరిగ్గా సమానం.

ఆ చివరి నియమం ముఖ్యమైనది: HLL ఒక ఉజ్జాయింపు అయినా, merge నిర్ధారితమైనది – కాబట్టి ఇక్కడ "దాదాపు సమానం" కాదు, సరిగ్గా సమానం అని పరీక్షించవచ్చు.

120 యాదృచ్ఛిక runs • 5,64,649 events పంపాం
నియమ ఉల్లంఘనలు: 0

ఏ దారులు నడిచాయి:

sent	5,64,649
dup	84,649
late	13,102
counted	4,66,898
shards	355

==== mutation testing ====

మార్పు లేని code	→ 0/120 విఫలం
ingest lo eventTime కి బదులు arrivalTime వాడితే	→ 50/50 విఫలం
ఉదా: event 1225: ఆశించినది late, వచ్చినది counted	
నకలు తనిఖీ తీసేస్తే	→ 50/50 విఫలం
ఉదా: event 3979: ఆశించినది dup, వచ్చినది late	
నకలుని event id కి బదులు వాడుకరి మీద చూస్తే	→ 50/50 విఫలం
ఉదా: event 3979: ఆశించినది dup, వచ్చినది late	
watermark పట్టించుకోకపోతే	→ 50/50 విఫలం
ఉదా: event 1225: ఆశించినది late, వచ్చినది counted	
HLL merge lo గరిష్ఠానికి బదులు కనిష్ఠం	→ 50/50 విఫలం
ఉదా: కలిపాక window 15: uniques 0 ≠ కేంద్రం 178	
combine lo uniques ని కూడితే (merge కాకుండా)	→ 50/50 విఫలం
ఉదా: కలిపాక window 15: uniques 48 ≠ కేంద్రం 178	

ఆరు మార్పులు, ఆరు పట్టుబడ్డాయి – మరియు చివరి రెండు ముఖ్యమైనవి

"HLL merge lo గరిష్ఠానికి బదులు కనిష్ఠం" → uniques 0. §11 lo చెప్పిన "గరిష్ఠం merge ని సాధ్యం చేస్తుంది" అనేది ఒక వివరణ కాదు – అది ఒక పరీక్షించదగిన ఆస్తి.

"combine lo uniques ని కూడితే" → $48 \neq 178$. ఇది §10 యొక్క 405% ఉబ్బు, ఒక mutation గా.

ఒక design నిర్ణయాన్ని ఒక mutation గా రాయగలిగితే, అది ఒక అభిప్రాయం కాదు.

13. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

నిమిషాలు	ఏమి చెప్పాలి	ఏ సంఖ్య చెప్పాలి
0-6	Requirements · ఇది ఒక invoice, ఒక dashboard కాదు	1.7 PB → 72 GB · 24,240 రెట్లు
6-13	At-least-once → నకళ్ళు	0.5% retry = 0.49% అదనపు బిల్లు
13-18	dedup విండో · దాని memory	విండో ≥ retry ఆలస్యం · 5 ని = 4.8 GB
18-26	event time vs arrival time · తేక	15 ని వేచి ఉన్నా 0.16% రాలేదు
26-32	ఆలస్యమైన దాన్ని ఏమి చేయాలి	"ఇప్పటి window" మెరుగ్గా కనిపిస్తుంది, 1,798 తప్పు గంటల్
32-40	ప్రత్యేక వాడుకరులు కూడబడరు	shards కలిపితే 405% ఉబ్బు
40-45	HLL · మరియు అది ఏమి చేయలేదు	16 KB vs 152 MB · తీసివేయలేము

ఈ PROBLEM LO మిమ్మల్ని వేరుగా నిలబెట్టే ఒక్క క్షణం

చాలామంది "ప్రత్యేక వాడుకరులకి HyperLogLog వాడతాను" అని చెప్పి ముందుకి వెళ్తారు. పేరు తెలుసు, ముగిసింది.

మీరు ఒక అడుగు ముందుకి: "HLL యొక్క నిజమైన విలువ memory ఆదా కాదు – అది merge చేయగలగడం. ప్రతి register ఒక గరిష్ఠం ఉంచుతుంది, మరియు గరిష్ఠం idempotent మరియు commutative. అందుకే shards ని ఏ క్రమంలో, ఎన్నిసార్లు కలిపినా అదే జవాబు. నేను కొలిచినప్పుడు, ప్రత్యేక సంఖ్యలను సాధారణంగా కూడితే 405% ఉబ్బు వచ్చింది – HLL merge తో అది సున్నా. మరియు అదే కారణానికి HLL తీసివేయలేదు: గరిష్ఠాన్ని వెనక్కి తిప్పలేము."

ఇది ఒక పనిముట్టు పేరు కాదు – ఆ పనిముట్టు ఎందుకు పనిచేస్తుందో, మరియు అదే కారణానికి ఎక్కడ పనిచేయడో మీకు తెలుసని చూపుతుంది.

14. నీటితో చెప్పాల్సిన English script

Framing గురించి:

"The aggregation itself isn't the hard part – a petabyte and a half of events a day compresses to about seventy gigabytes of answers, which is a twenty-four-thousand-fold reduction, and a stream processor does that on maybe a hundred machines. What makes this hard is that the output is an invoice, not a dashboard. Counting one percent high means overbilling advertisers; one percent low means underpaying publishers. So every approximation needs a number attached to it."

Duplicates గురించి:

"Kafka gives at-least-once, so retries duplicate events. I measured it: a half-percent retry rate inflates the count by half a percent – at eighty-six billion clicks a day that's four hundred thirty million phantom clicks. This isn't a bug, it's the delivery guarantee working correctly. So the application has to dedupe on an event id generated at the client. The window has to be at least as long as the producer's maximum retry delay – at sixty seconds I still had two point six percent inflation; at three hundred seconds it was exactly zero. And that window costs memory: at a million clicks a second, five minutes is about five gigabytes, and a full day is twenty-two machines just to answer 'have I seen this before'."

Late events గురించి:

"Then event time versus arrival time. A click on a phone in a tunnel arrives ten minutes later. I measured the arrival-delay tail: waiting one minute still misses one point nine percent, fifteen minutes misses point one six percent, two hours misses point zero one percent. It never reaches zero, so 'final' is a decision rather than a moment. The interesting part is what you do with a late event. Putting it in the current window looks *better* on every aggregate metric – total error dropped from one point nine percent to point four percent – but nine thousand events were in the wrong minute and eighteen hundred were in the wrong *hour*, which means billing them to a campaign that had already ended. So the right answer is restating the old window, and the cost is that your storage has to be mutable and your numbers provisional."

Uniques గురించి:

"Last, reach. Clicks are additive; unique users are not. When I sharded one ad across eight processors and summed their unique counts, I got four hundred five percent inflation – the same user counted eight times. Exact sets fix it and cost a hundred fifty megabytes per ad at ten million uniques. HyperLogLog gets it to sixteen kilobytes with about one percent error. But the reason I'd reach for it isn't the memory – it's that HLL merges, because each register holds a maximum and maximum is idempotent and commutative. And that same property is why it can't subtract, and why you can't ask whether a specific user is in it. So I'd bill on clicks, which are exact, and report reach from HLL, which is approximate."

15. Follow-ups – fraud, lambda vs kappa, సరిదిద్దడం

"Bot clicks ని ఏమి చేయాలి?"

అవి ఇప్పటికే మీ లెక్కలో ఉన్నాయి, మరియు అదే సమస్య. రెండు దశల విధానం: **తక్షణ filtering** (స్పష్టమైన సంకేతాలు – అసాధ్యమైన click రేటు, తెలిసిన data centre IP లు, user-agent) మరియు **తర్వాతి విశ్లేషణ** (ప్రవర్తన నమూనాలు, ఇది గంటలు పడుతుంది). రెండో దశ ఒక **సరిదిద్దే ప్రక్రియ** – అంటే §8 యొక్క "పాత window ని సరిదిద్దడం" అనే సామర్థ్యం ఇక్కడ మళ్ళీ అవసరం. **నేను fraud detection ని కొలవలేదు** – కానీ దాని architectural అవసరం స్పష్టం: మీ లెక్కలు మార్చదగినవిగా ఉండాలి.

"Lambda architecture అవసరమా?"

సాంప్రదాయిక జవాబు: ఒక **వేగవంతమైన stream** (తక్షణ, ఉజ్జాయింపు) మరియు ఒక **batch** (నెమ్మది, ఖచ్చితం), రెండింటినీ కలపడం. దీని సమస్య – **అదే logic ని రెండుసార్లు రాయడం**, మరియు అవి కాలక్రమేణా విడిపోతాయి.

Kappa ప్రత్యామ్నాయం: ఒక్కటే stream pipeline, మరియు సరిదిద్దాలంటే ముడి events ని **మళ్ళీ నడపడం** (Kafka lo అవి ఇప్పటికే ఉన్నాయి – HLD 13 §15 యొక్క retention). ఇది సరళమైనది, మరియు దీనికి మీ ముడి events ని ఎక్కువ కాలం ఉంచాలి – 17 TB/రోజు × 30 రోజులు = **510 TB**. అది object storage lo చౌక.

"ఒక నెల తర్వాత తప్పు కనుగొంటే?"

ఇదే నిజమైన ఉత్పత్తి ప్రశ్న, మరియు దీనికి జవాబు **ముడి events ఎంతకాలం ఉన్నాయి** అనేదాని మీద ఆధారపడుతుంది. కాబట్టి రెండు నిల్వలు: **సమాధానాలు** (72 GB/రోజు, శాశ్వతం) మరియు **ముడి events** (17 TB/రోజు, 30-90 రోజులు). ముడి events ఉన్నంతవరకు మీరు ఏ సంఖ్యనైనా తిరిగి లెక్కించగలరు. అవి పోయాక – ఆ నెల సంఖ్యలు **శాశ్వతంగా** అవే.

"ఈ doc lo నేను ఏమి కొలవలేదు"

మూడు విషయాలు: **fraud detection** (పైన), **stream processor యొక్క state recovery** (ఒక processor చనిపోతే dedup విండో మరియు పాక్షిక windows ఎక్కడి నుంచి తిరిగి వస్తాయి – checkpointing, మరియు దాని ఖర్చు), మరియు **impressions** (నేను clicks మాత్రమే simulate చేశాను; impressions 100 రెట్లు ఎక్కువ మరియు ఒక్కొక్కటి తక్కువ విలువైనది, కాబట్టి అక్కడ sampling ఒక చెల్లుబాటు అయ్యే ఎంపిక – clicks కి కాదు).

16. ఏమి నేర్చుకున్నాం

1. ఒక సంఖ్య ఒక invoice అయితే, "దాదాపు సరైనది" ఒక సమాధానం కాదు. 0.5% retry రేటు = **0.49%** అదనపు బిల్లు, రోజుకి 43 కోట్ల నకిలీ clicks.
2. At-least-once మద exactly-once లెక్క నిర్మించలేరు – application చేయాల్సిందే. మరియు dedup విండో producer యొక్క retry విధానంతో ఒక చెప్పని ఒప్పందం.
3. ప్రతి "గుర్తుపెట్టుకో" కి ఒక memory ధర ఉంది. 5 నిమిషాలు = 4.8 GB. ఒక రోజు = **22 machines**.
4. "తుది సంఖ్య" అనేది ఒక నిర్ణయం, ఒక క్షణం కాదు. 15 నిమిషాలు వేచి ఉన్నా 0.16% రాలేదు; 2 గంటలు వేచి ఉన్నా 0.01%. తేక ముగియదు.
5. ఒక సగటు తప్పు శాతం నష్టాన్ని దాచగలదు. "ఇప్పటి window lo వేయడం" యొక్క తప్పు **4 రెట్లు తక్కువ**, మరియు **1,798 clicks** తప్పు గంటలో.
6. కొన్ని కొలతలు కూడబడతాయి. కొన్ని కూడబడవు. Clicks కూడబడతాయి. ప్రత్యేక వాడుకరులు కాదు – shards కలిపితే **405%** ఉబ్బు.
7. HLL యొక్క విలువ memory కాదు, merge. ప్రతి register ఒక గరిష్ఠం, మరియు గరిష్ఠం idempotent మరియు commutative.
8. మరియు అదే కారణానికి అది తనివేయలేదు. ఒక పనిముట్టు ఎందుకు పనిచేస్తుందో తెలిస్తే, అది ఎక్కడ పనిచేయడో కూడా తెలుస్తుంది.

ఈ DOC నుంచి ఒక్క వాక్యం గుర్తుపెట్టుకోవాలంటే

ఈ problem lo ప్రతి విరుపూ ఒకే ప్రశ్నకి వేరే రూపం: "ఒకే విషయాన్ని రెండుసార్లు లెక్కపెట్టకుండా ఎలా ఉండాలి?"

§4 – అదే event రెండుసార్లు వచ్చింది → dedup.

§8 – అదే event రెండు windows lo పడింది (లేదా ఏ window lo నూ పడలేదు) → watermark మరియు సరిదిద్దడం.

§10 – అదే వాడుకరి రెండు shards lo ఉన్నాడు → HLL merge.

మూడూ "ఒక్కసారే" అనే హామీ గురించి, మరియు మూడింటికీ వేర్వేరు పరిష్కారాలు – ఎందుకంటే "ఒకటే" అనే మాటకి మూడు వేర్వేరు అర్థాలు: **ఒకే సందేశం, ఒకే క్షణం, ఒకే వ్యక్తి**.